Monk: Chord-Ahead Text Entry with Probabilistic Word Inference

Lei (Lorin) Zhao

July 2026

Abstract

We present Monk, a text-entry method for standard keyboards in which the user presses several key letters of a word simultaneously — a *chord* — and the system infers the intended word from the chord and the surrounding context. Monk generalizes sequential typing: any prefix of skill, from ordinary letter-by-letter typing to fully chorded entry, produces correct text, and the two styles can be mixed freely inside a single word. We formalize chords as ordered sequences of unordered key groups, give a log-linear inference model over a frequency lexicon with embedded on-device neural re-ranking (a 135M-parameter language model), and derive the method's speed ceiling from a keystroke-level motor model. On frequency lexicons of six languages, chorded entry reduces expected key presses per word by 21–25% before any context is used, and a motor-time analysis shows a practical ceiling near twice the user's sequential typing rate. We describe the disambiguation interface, an adaptive time-window classifier separating chords from fast sequential typing, and a user adaptation rule that converges after a single correction.

1. Introduction

A touch typist producing English text executes roughly 5.7 keystrokes per word (4.7 letters plus a space), strictly serially. Every improvement in typing speed over the last century — from QWERTY alternatives to autocorrect — has attacked the *cost of each keystroke* or the *number of keystrokes*, but not the serial constraint itself. Court stenographers broke the constraint long ago: a stenotype machine enters an entire syllable or word in one simultaneous stroke, which is why machine stenography reaches 225 words per minute while the fastest QWERTY typists plateau near 120. Stenography, however, demands special hardware and months of training on an opaque phonetic code.

Monk brings the stenographic principle — *one stroke, one word* — to the ordinary keyboard, with an encoding every user already knows: the letters of the word itself. To write "apple" the user presses some informative subset of its letters together

(`A+P`, `A+P+L`, `A+P+L+E` …) and commits with the space bar. The system infers the word. When inference is uncertain, a candidate bar appears under the caret and the user selects with a digit, arrow keys, or space. Three properties make this practical:

1. **Graceful generalization.** A full sequentially-typed word is just the longest possible chord sequence; it always commits to itself. Users mix chorded and normal typing freely, word by word, with no mode switch.
2. **Self-revealing encoding.** The chord for a word is (a subset of) its own spelling. There is no code to memorize, only a habit to acquire.
3. **Contextual disambiguation.** A language model — from unigram frequencies up to an LLM — supplies the prior that makes short chords unambiguous in practice, in exactly the situations where humans find them unambiguous.

1.1 Related work

T9 and its successors map one keypress per letter onto a reduced keyset and disambiguate with a dictionary; the keystroke count is unchanged. Gesture keyboards (ShapeWriter, Swype) draw one continuous stroke per word on a touchscreen — the closest interaction ancestor of Monk, but inapplicable to physical keyboards. Dasher restructures entry around continuous pointing. Abbreviation expanders (and modern "text replacement") require the user to pre-register each abbreviation. Stenotype and systems such as Plover achieve simultaneity with a phonetic chord code on dedicated or n-key-rollover hardware. Monk occupies the unclaimed cell of this matrix: standard hardware, no learned code, simultaneous entry, probabilistic decoding.

1.2 Why "Monk"

The system is named for Thelonious Monk (1917–1982), the jazz pianist and composer. The homage is not decorative; it is a description of the method. Monk's playing was defined by *chords over runs* — stabbed, dissonant voicings placed exactly once — and, above all, by *economy*: he was famous for the notes he left out, trusting the harmonic context to make the listener hear them anyway. That is precisely this keyboard's contract with its user. You strike a few letters of a word as one chord, deliberately leaving most of them out, and the language model — the harmonic context — supplies what was implied. A dissonant chord in Monk's hands resolves in context; an ambiguous chord under your hands resolves in context too. Monk is credited with saying that "the piano ain't got no wrong notes"; on this keyboard, neither does the chord — inference and the candidate bar absorb the imprecision. Fewer notes, more music.

2. Interaction model

2.1 Chords and groups

Modern keyboards report each key press as a separate event with a timestamp; pressing several keys "at once" yields a burst of events a few milliseconds apart (rollover). We therefore define entry not by simultaneity but by temporal clustering. Let key events arrive at times $t_1 < t_2 < \dots$ Two consecutive events belong to the same **group** iff $t_{i+1} - t_i < \tau$, where τ is the chord window (§5.3). A **chord sequence** is the resulting ordered list of groups $c = (g_1, \dots, g_m)$: order *within* a group is unreliable (rollover scrambles it), order *across* groups is reliable. Sequential typing is the special case where every group is a singleton.

2.2 The commit protocol

- **Letters** accumulate in an underlined composition buffer at the caret.
- **Space** requests inference. If the buffer, typed sequentially, is itself a lexicon word, it commits unchanged (the *identity property*). Otherwise the top-ranked candidate commits, followed by a space.
- **Ambiguity.** If the top two candidates score within a margin δ, the system does not guess: a horizontal candidate bar appears beneath the caret. The user selects with `1`–`9`, arrow keys and `Return`, or presses space again to accept the highlighted candidate. This is the rare-case interface requested of the design: it appears only when the chord genuinely underdetermines the word.
- **Return** commits the buffer literally; **Escape** likewise abandons inference. Punctuation resolves the buffer and then passes through.

The identity property is the load-bearing UX decision: Monk is *strictly additive* over ordinary typing. A user who never chords loses nothing; a user who chords badly falls back to the bar; a user who chords well types words in single strokes.

2.3 Flow mode

The bar is the right default, but it is an interruption, and §5.4 shows the interruption rate β is the dominant tax on throughput. **Flow mode** removes it: the bar never appears and every commit takes the highest-ranking candidate. On an ambiguous chord the decision is not left to the unigram model alone — the embedded neural re-ranker (§3.4) is consulted *synchronously*, a single bounded inference measured at 35–60 ms on Apple Silicon, below the perceptual threshold of a keystroke echo. In effect flow mode trades the margin rule of §3.3 ($\delta = 1.5$) for $\delta = 0$ plus a stronger prior at the decision point. The failure mode is honest: a wrong guess is an ordinary typo, repaired by retyping — which simultaneously teaches the adaptation table (§6), so each personal chord fails at most once. Flow mode is therefore best enabled after a short adaptation period, and the toggle (in the input menu, persisted in configuration) makes it a per-mood choice rather than a commitment.

3. Inference

3.1 Match predicate

Fold case and diacritics; let w be a lexicon word and $c = (g_1, \ldots, g_m)$ a chord sequence with letters $\ell(c)$, $|\ell(c)| = k$. c **matches** w iff:

1. the first letter of w is a member of g_1, and
2. the letters of c embed into w as a subsequence that respects group order, letters within a group being free to permute.

The embedding is computed greedily: within a group, repeatedly consume the letter whose next occurrence in w is earliest. The greedy choice is optimal here because consuming the earliest-available occurrence never blocks a later letter that a different order would have permitted.

Condition (1) generalizes "the chord starts the word": because rollover can scramble even the first group, *any* letter of g_1 may serve as the word's initial, and the scoring function (below) rewards the chronologically first key when it does.

3.2 Scoring

Ideal inference is $\hat{w} = \arg\max_w P(w \mid \text{context})\, P(c \mid w)$. Monk approximates the log-posterior with a log-linear score over cheap, interpretable features:

$$s(w, c) = \log_2 f(w) + \sum_j \lambda_j \phi_j(w, c)$$

feature ϕ_j	**λ_j**	**rationale**
buffer spells w exactly (unchorded only)	+40	identity property; dominates everything
w starts with the chronologically first key	+4	first contact is usually the intended initial
arrival order already embeds in w	+2.5	unscrambled chords carry order information
first $\min(3, k)$ letters all fall in w's first 4	+2	chords are drawn from word onsets
last chord letter equals last letter of w	+3	word endings are salient to users
per letter of w not covered by c	−0.22	mild preference for tighter matches
user previously chose w for this chord (n times)	$+6\min(n, 3)$	adaptation, §6

$f(w)$ is the corpus frequency. The exact-match bonus is *disabled when the input is chorded*: a simultaneous press never means the literal letter string, which keeps corpus debris ("th", "bc") from shadowing real words.

The weights were tuned on the match statistics of §5 to satisfy three ordering constraints: (i) identity beats all inference for sequential input; (ii) an onset-ordered chord beats a higher-frequency word that matches only under permutation; (iii) one user correction (+6) outweighs a typical top-two frequency gap within a chord's candidate set (the median gap is 3.9 bits, and 90% of gaps are under 6 bits), so adaptation wins after a single selection.

3.3 Ambiguity and the candidate bar

Let $s_1 \geq s_2$ be the two best scores. Monk commits silently iff $s_1 - s_2 \geq \delta$ with $\delta = 1.5$ bits; otherwise it shows the bar. δ trades silent-error rate against bar-interruption rate: at $\delta = 0$ every chord commits instantly but near-ties are coin flips; as $\delta \to \infty$ every chord interrupts. Because scores are calibrated in log-frequency bits, $\delta = 1.5$ means "commit only if the winner is at least $2^{1.5} \approx 2.8\times$ more probable than the runner-up," a direct bound on the silent-error odds of $1/(1 + 2^{\delta}) \approx 26\%$ *in the worst accepted case* and far lower in expectation.

3.4 On-device neural re-ranking

The chord engine is a sound retriever: the correct word is in its candidate list whenever the chord matches it. Context selection is where a neural language model helps, and the candidate-scoring task is small enough that a very small model suffices: the model never generates — it only compares the likelihoods of half a dozen given words as continuations of the user's last few words. Monk embeds SmolLM2-135M (Apache-2.0), a 135-million-parameter causal LM, 4-bit quantized to ~100 MB and executed by llama.cpp compiled into the input method. When the bar appears (and only then), each candidate w with tokenization $u_1..u_r$ is scored by its continuation log-probability

$$\log P(w \mid \text{context}) = \sum_{j=1}^{r} \log P_{\theta}(u_j \mid \text{context}, u_{<j})$$

and the bar reorders by this score if inference finishes before the user selects. On Apple Silicon (CPU-only) a full re-rank of six candidates over a twelve-word context measures 33–45 ms — comfortably inside the human decision time the bar itself represents. The architecture keeps three guarantees: the model is never on the critical typing path; it cannot introduce a word outside the chord-matched set (no hallucination surface); and no text ever leaves the machine. With this contextual prior, chords like `A+P` after "she bit into the" rank "apple" first — the unigram model alone cannot know that.

4. Why chords are informative enough: an entropy argument

A lexicon of the 10,000 most frequent words carries at most $\log_2 10^4 \approx 13.3$ bits per word, and the frequency-weighted entropy of such a lexicon is far lower — about 9–10 bits for subtitle-domain English. Now count the information in a chord of k letters: the first letter (≈ 4.1 bits given English letter statistics at word onsets), each additional letter with approximate positional order (≈ 3–4 bits each), plus a strong end-of-word signal when the user includes the final letter. A 3-letter chord therefore delivers on the order of 11–12 bits — already comparable to the lexicon's entropy, which is why the majority of words resolve at $k \leq 4$ (Table 1), and why adding *context* (which contributes several further bits of prior) pushes short chords over the threshold.

Table 1 — Minimal chord length at which each of the top 10,000 English words becomes the unique frequency argmax (chord = word prefix of length k; no context; measured on the shipped lexicon):

k	words	cumulative
≤2	243	2.4%
3	1,004	12.5%
4	2,447	36.9%
5	2,811	65.0%
6	1,681	81.9%
≥7	1,814	100%

Frequency weighting matters: common words resolve much earlier than rare ones, so the *expected* chord length per running word is only **2.90 keys**, versus 3.80 letters for full typing (23.6% fewer key presses, before counting the temporal collapse of those presses into strokes).

5. Mathematical optimization of the design

5.1 Keystroke-per-word analysis across languages

Running the same analysis on six frequency lexicons (OpenSubtitles-derived, top 10,000 words each):

language	keys/word (chord)	keys/word (full)	savings	unresolved at full length
English	2.90	3.80	23.6%	20.2%
Spanish	3.26	4.13	21.0%	22.6%

language	keys/word (chord)	keys/word (full)	savings	unresolved at full length
French	3.10	3.95	21.6%	23.8%
German	3.31	4.39	24.5%	22.1%
Italian	3.33	4.30	22.5%	20.8%
Portuguese	3.25	4.20	22.5%	21.1%

"Unresolved" words are those never becoming the unique argmax even fully spelled (they are dominated by a more frequent word matching the same chord); these are exactly the cases the candidate bar and the context model exist for. The uniformity across languages — savings within a 3.5-point band — suggests the result is a property of Zipfian lexicons generally, not of English.

5.2 The speed ceiling: a motor model

Let a typist's inter-keystroke interval be t_k (at 60 WPM English, $t_k \approx 175$ ms). Sequential entry of an average word costs $(4.7 + 1)\, t_k \approx 1.0$ s. For chorded entry, the motor literature on piano and stenotype gives the cost of a prepared multi-finger stroke as roughly 1.2–1.5 t_k regardless of finger count within one hand-shape. An average Monk word is then one chord stroke plus one space:

$$T_{\text{comp}} \approx (1.35 + 1)\, t_k \approx 0.41 \text{ s} \Rightarrow \approx 145 \text{ WPM at the same finger speed.}$$

Two taxes reduce the ceiling. First, long or rare words need a second group (chord–then–letters), adding $\approx t_k$ each; with the Table 1 distribution this costs about 12% on average. Second, the bar: if a fraction β of words interrupt with a selection costing $t_{\text{sel}} \approx 3.5\, t_k$ (read + press digit), throughput divides by $1 + \beta\, t_{\text{sel}}/T_{\text{comp}}$. With the context-free $\beta \approx 0.2$ the ceiling is ≈ 105 WPM; an LLM prior that halves β restores ≈ 125 WPM, and user adaptation drives the *personal* β toward zero on each user's own vocabulary. The design conclusion: **the bar rate β, not the chord rate, is the quantity to optimize** — which is why Monk spends its complexity budget on ranking (features, adaptation, LLM) rather than on richer chord syntax.

5.3 The chord window as a two-class classifier

The grouping threshold τ classifies each inter-key gap as *within-chord* or *between-strokes*. Empirically the two gap populations are well separated: rollover gaps concentrate below 30 ms, while deliberate sequential gaps for fluent typists exceed 100 ms even at speed. Modeling the populations as log-normal with $(\mu_1, \sigma_1) = (20$ ms, 1.6) and $(\mu_2, \sigma_2) = (160$ ms, 1.5), the Bayes boundary lands between 45 and 60 ms across a wide range of mixing priors. Monk ships $\tau = 45$ ms (configurable), biased low because the two error types are asymmetric: a missed chord (split into two groups) usually still matches the word — group order is preserved — while a false merge (two intended letters fused into one unordered group) discards order

information. The classifier can be made adaptive by tracking each user's sequential-gap distribution and re-solving for the boundary; the shipped engine exposes the threshold in its configuration for exactly this purpose.

5.4 Maximum theoretical time saving

The keystroke figures of §5.1 understate the real prize, because chorded letters are not merely fewer — they are *simultaneous*. Here we bound the total typing-time saving.

Upper bound. From the motor model of §5.2, a sequential English word costs $(4.7 + 1)\,t_k = 5.7\,t_k$ (letters plus space), while a perfectly chorded word costs one prepared stroke plus a space, $(1.35 + 1)\,t_k = 2.35\,t_k$. The **maximum theoretical time saving is therefore**

$$1 - \frac{2.35\,t_k}{5.7\,t_k} \;=\; 58.8\% \;\approx\; \mathbf{59\%},$$

independent of the typist's speed t_k — a shade under "type in less than half the time." It is an upper bound: it assumes every word resolves from a single chord with no candidate bar.

Practical estimate. Charging the taxes of §5.2 — a second chord group on long or rare words (+12% of stroke time, from the Table 1 length distribution) and a candidate-bar interruption on a fraction $\beta \approx 0.1$ of words (achievable with the neural context prior plus user adaptation) at $3.5\,t_k$ per interruption — the expected word cost rises to $\approx 3.0\,t_k$, for a **practical saving of ≈ 47%**. We quote the band **45–50%** to cover β between 0.08 and 0.15.

What the percentage means in hours. Take a 40 WPM baseline — a typical professional typist — and the theoretical 59% / practical 47% band:

task	words	typing time today	with Monk	time returned
a substantial email	100	2.5 min	1.0–1.3 min	~1.2–1.5 min
an hour of email per day	~2,400/day	60 min/day	25–32 min	**~30 min every day**
a college essay	2,000	50 min	21–27 min	~25 min
a novel (NaNoWriMo)	50,000	20.8 h	8.5–11 h	~10–12 h
the complete Harry Potter series	1,084,170	452 h	185–240 h	**210–265 h ≈ 27–33 working days**

The last row is the one to sit with: the raw typing of the Harry Potter series (1,084,170 words across seven books) costs about 452 hours of pure keystroking at 40 WPM; at Monk's ceiling, roughly 265 of those hours — more than six working weeks — never need to happen. For an ordinary knowledge worker who types about

two hours a day, the practical band returns **≈ 55 minutes a day, or six working weeks per year**. These figures cover transcription time only; composition (thinking) time is untouched — which is also why the saving matters: Monk removes time from the mechanical part of writing and returns it to the part that was never the keyboard's business.

5.5 Where the chord letters should come from

Given the freedom to press any k letters of a word, which letters maximize disambiguation? Ranking letter positions by conditional information gain over the lexicon shows the ordering: **first letter ≫ last letter > early strong consonants > vowels**. The first letter alone partitions the lexicon into buckets whose sizes vary by two orders of magnitude; the last letter is the next most informative because English inflection concentrates there; interior consonants beat vowels because consonants are less predictable given their neighbors. This yields the teaching heuristic used throughout the trainer: *first + strongest middle consonant + last* ("k-b-d" → "keyboard"), which is also the chord the scoring features (§3.2) are shaped to reward.

6. Learning and personalization

Every bar selection and every silent correction updates a per-user table mapping the normalized chord (groups sorted internally) to the chosen word. The boost +6min $(n, 3)$ is a capped MAP-style prior: after one selection the user's choice outranks the frequency winner in the median ambiguous set (§3.2 constraint iii); after three it outranks 99% of them; the cap prevents a runaway prior from hiding a genuinely intended different word forever. The table lives locally (`~/Library/Application Support/Monk/`), is human-readable JSON, and is the entire personalization state — deleting it resets the keyboard.

7. User-experience considerations

- **No mode, no penalty.** The identity property (§2.2) means Monk can be left enabled permanently; it only ever *adds* an interpretation to input that would otherwise be a non-word.
- **Visibility of system state.** The composition buffer is always underlined at the caret; inference happens at an explicit user action (space), never on a timer, so text never changes "by itself."
- **Cheap error recovery.** Before space: backspace edits the buffer. At space: the bar catches ambiguity. After a wrong silent commit: the word is selected-and-retyped like any typo, and the correction teaches the engine.
- **Progressive disclosure of skill.** The trainer (§8) sequences two-key chords → three-key chords → phrases; each level's chords are generated by the same heuristic the engine rewards, so practice transfers directly.
- **Latency discipline.** Local inference over a 25k lexicon is sub- millisecond on 2020s hardware (a first-letter index bounds each query to a few hundred

subsequence checks). The neural re-ranker is quarantined to the ambiguous path, runs asynchronously in ~35 ms, and never blocks a keystroke.

8. Implementation

The reference implementation ships as: (a) a macOS input method (Swift, InputMethodKit) installable per-user without administrator rights — it works in every text field system-wide, with SmolLM2-135M embedded via statically linked llama.cpp for offline contextual re-ranking; (b) a browser-based trainer implementing the identical engine and scoring in JavaScript, structured as a game with five "sets" (Warm-Up, Duets, Trio, Standards, Improv), live WPM / keystrokes-saved / streak meters, and per-language progress; (c) frequency lexicons for English, Spanish, French, German, Italian, and Portuguese with diacritic-folded matching. Both engines share the constants of §3.2 verbatim, so trained intuition transfers exactly from the game to the keyboard.

9. Limitations and future work

The silent-commit path currently uses only the unigram-plus-features model; extending the embedded neural re-ranker from the ambiguous path to a pre-emptive prior on every commit (its ~35 ms budget permits this between keystrokes) would cut the bar rate itself, not just resolve it better. Scripts without alphabetic spelling (Chinese, Japanese) need a composition layer — chording over pinyin or romaji is a natural extension, since both are Latin encodings with strong lexicon priors. Simultaneity detection on membrane keyboards with limited rollover caps chord size around 4–5 keys; this matches the useful chord-length distribution (Table 1) but should be validated per device. Finally, the motor-model ceiling of §5.2 is a prediction; a longitudinal user study measuring the learning curve against the trainer is the natural next step.

10. Conclusion

Monk shows that the stenographic speed principle survives translation to commodity keyboards if — and only if — the decoding burden moves from the user to a probabilistic engine. The chord code that requires no learning (the word's own letters) is informative enough, by the entropy accounting of §4, once a frequency prior and light context are added; the interaction contract that requires no trust (identity fallback, explicit commit, visible ambiguity) makes the inference safe to live inside every text field. Type the chord; the keyboard comps the rest.

Acknowledgments

The design, mathematical analysis, implementation, and drafting of this paper were carried out in collaboration with **Claude Fable 5** (Anthropic), used as an AI research and engineering assistant under the direction of the human author, who takes responsibility for the work. This disclosure follows the prevailing editorial norm that AI systems are acknowledged for their contributions rather than listed as authors. The frequency lexicons derive from the FrequencyWords compilation of the OpenSubtitles corpus; the embedded re-ranking model is SmolLM2-135M (Hugging Face TB research team), executed with llama.cpp.

References

1. C. E. Shannon. “Prediction and Entropy of Printed English.” *Bell System Technical Journal*, 1951.
2. S. K. Card, T. P. Moran, A. Newell. *The Psychology of Human-Monkuter Interaction* (the Keystroke-Level Model). Erlbaum, 1983.
3. P.-O. Kristensson, S. Zhai. “SHARK²: A Large Vocabulary Shorthand Writing System for Pen-based Monkuters.” *UIST*, 2004.
4. D. J. Ward, A. F. Blackwell, D. J. C. MacKay. “Dasher — a Data Entry Interface Using Continuous Gestures and Language Models.” *UIST*, 2000.
5. I. S. MacKenzie, R. W. Soukoreff. “Text Entry for Mobile Monkuting: Models and Methods, Theory and Practice.” *Human-Monkuter Interaction*, 2002.
6. The Open Steno Project, *Plover*. https://www.openstenoproject.org/
7. P. Lison, J. Tiedemann. “OpenSubtitles2016: Extracting Large Parallel Corpora from Movie and TV Subtitles.” *LREC*, 2016. (Source of the frequency lexicons, via the FrequencyWords compilation.)
8. L. Ben Allal et al. “SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model.” 2025. (The embedded re-ranking model, Apache-2.0.)
9. G. Gerganov et al. *llama.cpp* — LLM inference in C/C++. https://github.com/ggml-org/llama.cpp (The embedded inference runtime.)